



POSTMORTEM → REGRESSION GUARD

Never Again

Every postmortem becomes a guard your code can't forget.

Built with IBM Bob 2.0
github.com/harshgitty58/never-again-guard

Team Kira

Harsh Dhavane · TY Computer Engineering, VIT Pune



Postmortem action items drift, and the same bug comes back.

Items live in a doc

"Add a test", "check similar code", "update the runbook". They get tracked as tickets and are rarely closed.

"We added a test"

Nothing proves that test would have failed on the broken code. A test that always passes guards nothing.

Same mistake, other files

The root-cause pattern is almost always copy-pasted elsewhere, and nobody goes looking.

Incomplete action items make recurrence more likely (Google SRE workbook, postmortem culture).

Prove, don't claim. Bob generates; a deterministic verifier judges.

@ inc-102-bug

EXPECT FAIL

```
$ vitest run regression.test.js
X FAIL failedOn: assertion
  expected 10 to be less than or equal to 4
exit 1 · 1.1 s
```

@ inc-102-fix

EXPECT PASS

```
$ vitest run regression.test.js
✓ PASS all assertions hold

exit 0 · 3.2 s
```

Same test, two throwaway git worktrees. A crash or import error on the bug commit doesn't count. Anti-cheat rejects skipped, assertion-free or self-mocking tests.

One postmortem in, four verified outputs out.

1 · Regression test

Fails on an assertion at the bug commit and passes at the fix commit. Proven both ways.

2 · Variant rule

A Semgrep rule generalized from the root cause. It must match the bug and not the fix, then it scans the whole app.

3 · Variant fixes

Bob fixes every variant the rule finds. The verifier reruns until none are open.

4 · Action-item audit

Every item is marked guarded / partial / unguarded / process-only, with file evidence. No invented code for runbooks.

Bob does the generating. The judging is deliberately not AI.

01

Postmortem

Markdown: root cause, trigger, seed, action items.

Document
understanding

02

Bob reads

Guided by the never-again skill, pulls out the root cause, seed location and refs.

Agent mode · Skill

03

Guard generated

test + rule + guard.json per incident.

Agent mode

04

Verifier proves

Worktrees at both commits, anti-cheat, rule sanity checks, variant hunt.

No AI

05

Variants fixed

Bob fixes them, then the verifier reruns until exit 0.

Agent mode

Output: `reports/INC-xxx.json` → static 3D dashboard on Vercel. Guardrails live in the project rules (AGENTS.md): never edit app source while guarding, and always quote the verifier's real verdict.

PROOF ON THE DEMO APP

Run 1 found 7 hidden variants. Run 2: 0 open.



run 1 · 3 PARTIAL · 7 open variants



run 2 · 3 GUARDED · 0 open · verifier exit 0

IMPACT : ALL NUMBERS FROM VERIFIER REPORTS

Evidence instead of claims.

3/₃

incidents GUARDED:
time zone, retry storm,
₹NaN checkout

7 → 0

hidden variants found by
the rules, then fixed and
proven closed

11/₁₁

code action items with
file evidence (6/11
before); 4 honestly
process-only

<1 min

full verification of all
incidents, also on a fresh
clone

One IBM Bob task (39.26 Bobcoins) built the app, the postmortems, the verifier and all three guards, then fixed every variant.

From a hackathon demo to part of every incident's follow-up.

GitHub Action

Run the guard on every merged postmortem PR, and block merges that reintroduce a guarded pattern.

Import from incident tools

Pull postmortems and action items from incident.io and Jira instead of markdown only.

Follow-up automation

Schedule process-only action items (runbooks, on-call changes) with watsonx Orchestrate.

More languages

Rule packs beyond JavaScript: Python, Java and Go via Semgrep.